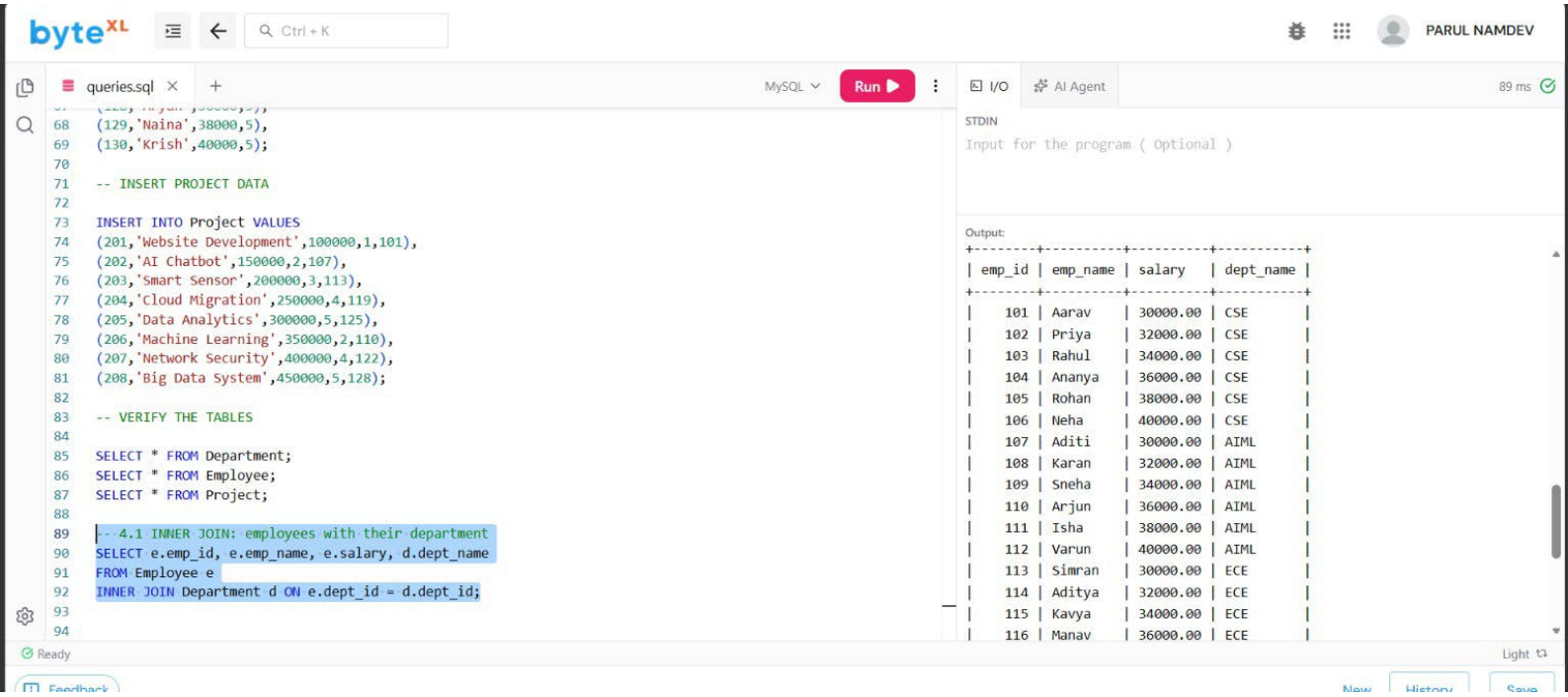
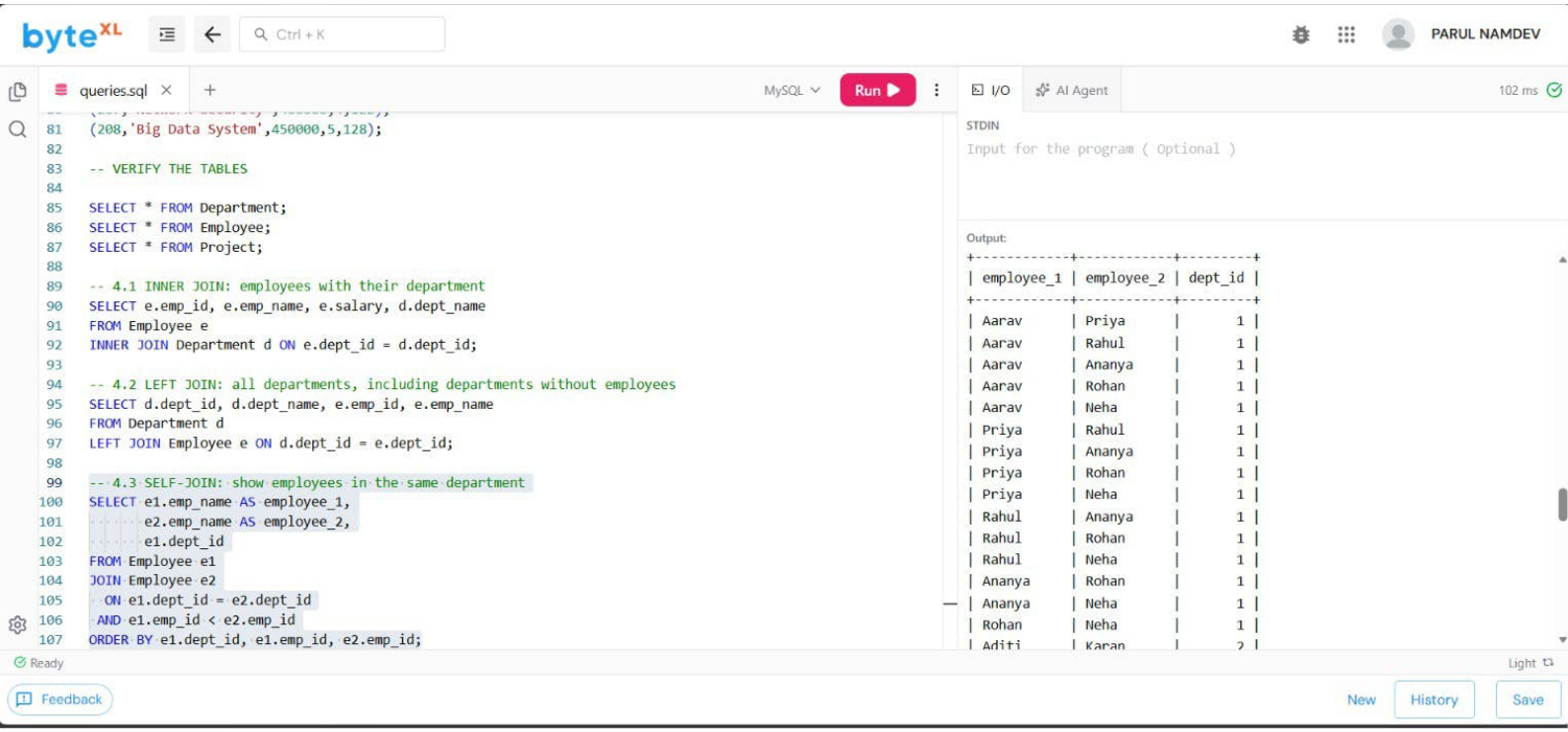
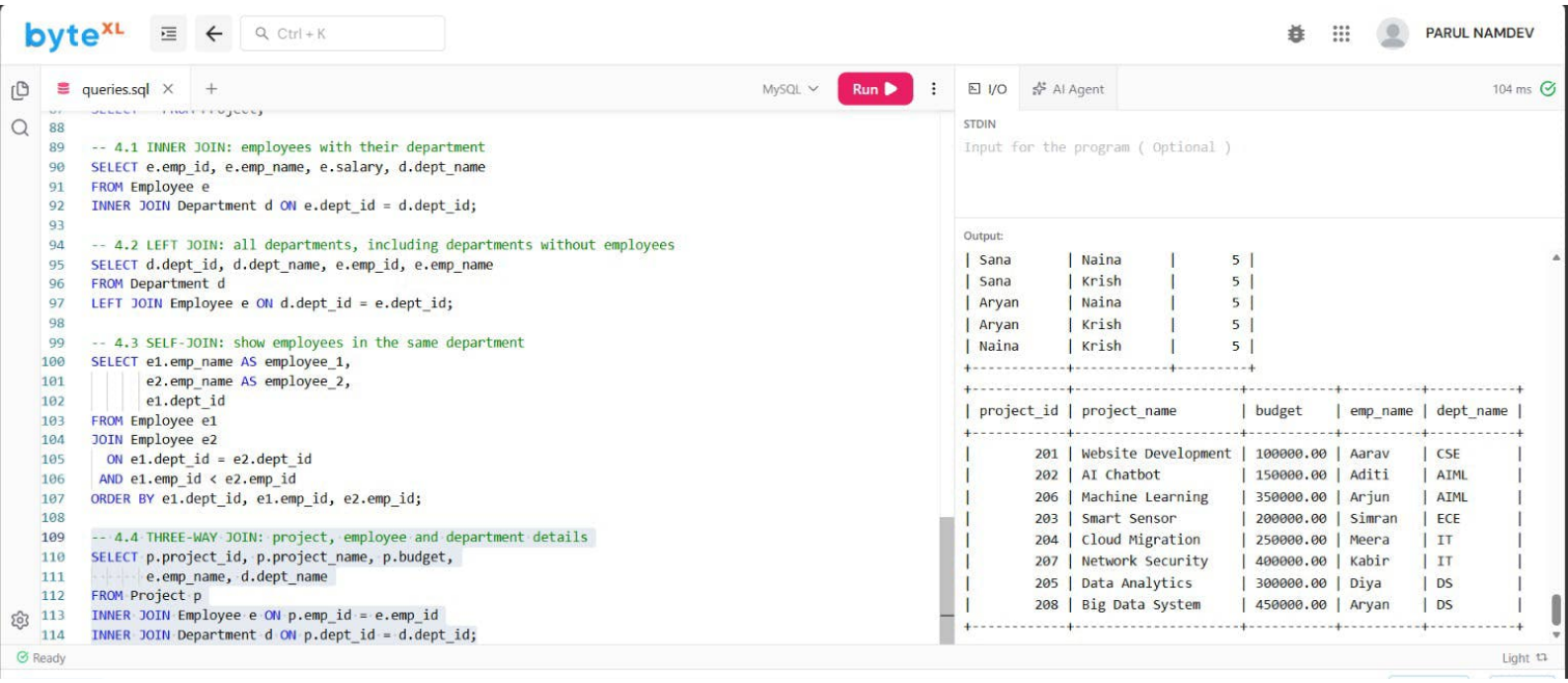
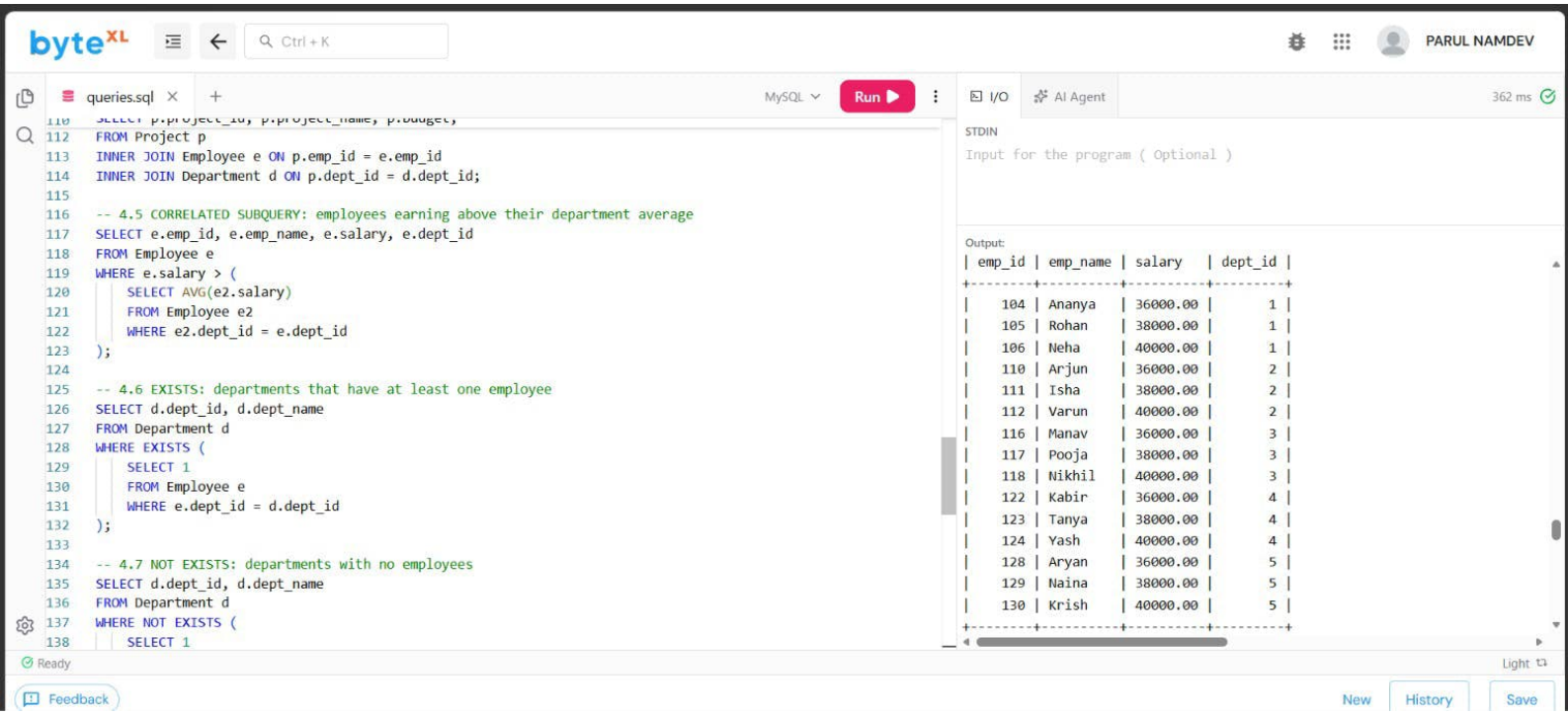

byteXL
Ctrl + K
PARUL NAMDEV
queries.sql
MySQL
Run
I/O
AI Agent
89 ms
STDIN
Input for the program (Optional)
Output:
(129,'Naina',38000,5),
(130,'Krish',40000,5);
-- INSERT PROJECT DATA
INSERT INTO Project VALUES
(201,'Website Development',100000,1,101),
(202,'AI Chatbot',150000,2,107),
(203,'Smart Sensor',200000,3,113),
(204,'Cloud Migration',250000,4,119),
(205,'Data Analytics',300000,5,125),
(206,'Machine Learning',350000,2,110),
(207,'Network Security',400000,4,122),
(208,'Big Data System',450000,5,128);
-- VERIFY THE TABLES
SELECT * FROM Department;
SELECT * FROM Employee;
SELECT * FROM Project;
-- 4.1 INNER JOIN: employees with their department
SELECT e.emp_id, e.emp_name, e.salary, d.dept_name
FROM Employee e
INNER JOIN Department d ON e.dept_id = d.dept_id;
emp_id	emp_name	salary	dept_name
101	Aarav	30000.00	CSE
102	Priya	32000.00	CSE
103	Rahul	34000.00	CSE
104	Ananya	36000.00	CSE
105	Rohan	38000.00	CSE
106	Neha	40000.00	CSE
107	Aditi	30000.00	AIML
108	Karan	32000.00	AIML
109	Sneha	34000.00	AIML
110	Arjun	36000.00	AIML
111	Isha	38000.00	AIML
112	Varun	40000.00	AIML
113	Simran	30000.00	ECE
114	Aditya	32000.00	ECE
115	Kavya	34000.00	ECE
116	Manav	36000.00	ECE
Ready
Light
Feedback
New
History
Save

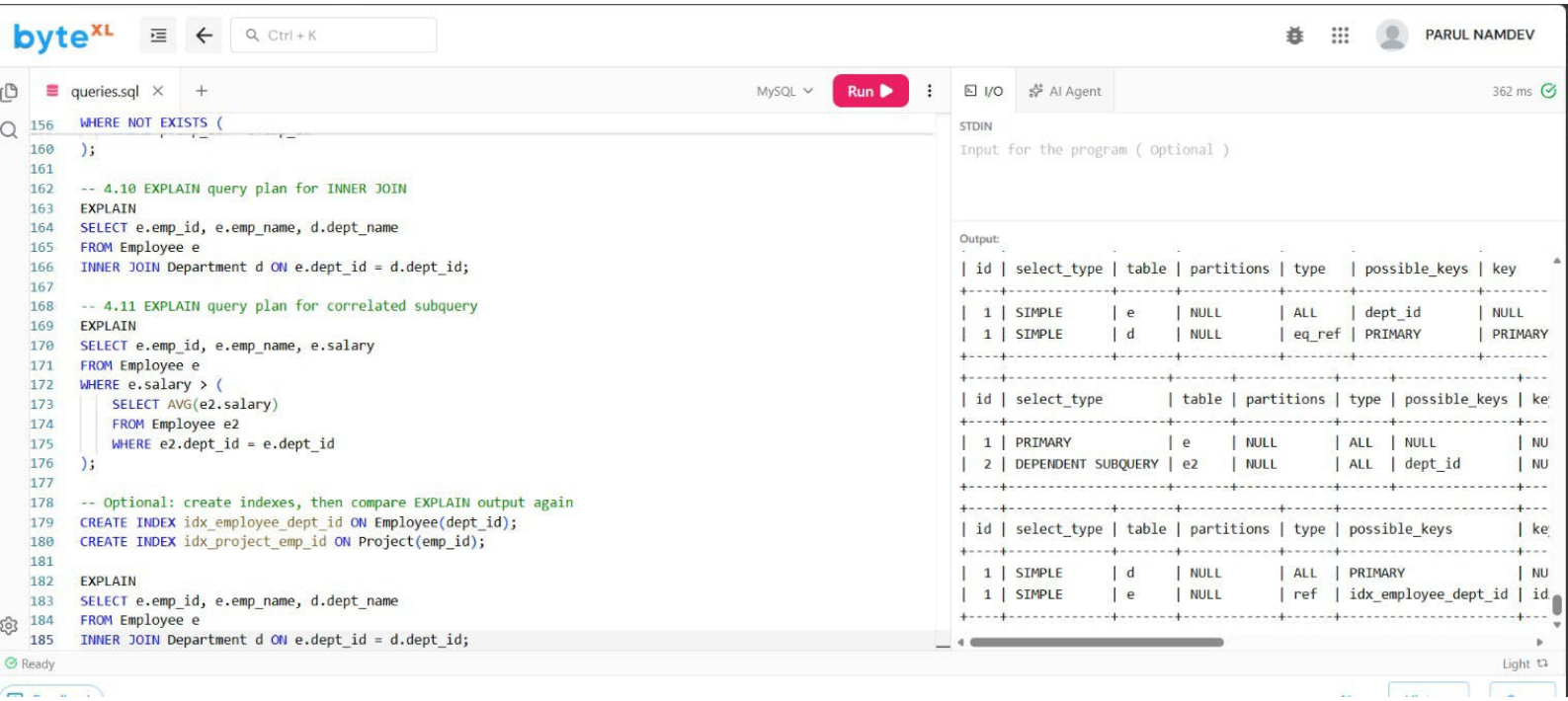

byteXL
Ctrl + K
PARUL NAMDEV
queries.sql
MySQL
Run
I/O
AI Agent
125 ms
STDIN
Input for the program (Optional)
Output:
-- INSERT PROJECT DATA
INSERT INTO Project VALUES
(201,'Website Development',100000,1,101),
(202,'AI Chatbot',150000,2,107),
(203,'Smart Sensor',200000,3,113),
(204,'Cloud Migration',250000,4,119),
(205,'Data Analytics',300000,5,125),
(206,'Machine Learning',350000,2,110),
(207,'Network Security',400000,4,122),
(208,'Big Data System',450000,5,128);
-- VERIFY THE TABLES
SELECT * FROM Department;
SELECT * FROM Employee;
SELECT * FROM Project;
-- 4.1 INNER JOIN: employees with their department
SELECT e.emp_id, e.emp_name, e.salary, d.dept_name
FROM Employee e
INNER JOIN Department d ON e.dept_id = d.dept_id;
-- 4.2 LEFT JOIN: all departments, including departments without employees
SELECT d.dept_id, d.dept_name, e.emp_id, e.emp_name
FROM Department d
LEFT JOIN Employee e ON d.dept_id = e.dept_id;
dept_id	dept_name	emp_id	emp_name
1	CSE	101	Aarav
1	CSE	102	Priya
1	CSE	103	Rahul
1	CSE	104	Ananya
1	CSE	105	Rohan
1	CSE	106	Neha
2	AIML	107	Aditi
2	AIML	108	Karan
2	AIML	109	Sneha
2	AIML	110	Arjun
2	AIML	111	Isha
2	AIML	112	Varun
3	ECE	113	Simran
3	ECE	114	Aditya
3	ECE	115	Kavya
3	ECE	116	Manav
Ready
Light


byteXL
Ctrl + K
PARUL NAMDEV
queries.sql
MySQL
Run
I/O
AI Agent
102 ms
STDIN
Input for the program (Optional)
Output:
(208,'Big Data System',450000,5,128);
-- VERIFY THE TABLES
SELECT * FROM Department;
SELECT * FROM Employee;
SELECT * FROM Project;
-- 4.1 INNER JOIN: employees with their department
SELECT e.emp_id, e.emp_name, e.salary, d.dept_name
FROM Employee e
INNER JOIN Department d ON e.dept_id = d.dept_id;
-- 4.2 LEFT JOIN: all departments, including departments without employees
SELECT d.dept_id, d.dept_name, e.emp_id, e.emp_name
FROM Department d
LEFT JOIN Employee e ON d.dept_id = e.dept_id;
-- 4.3 SELF-JOIN: show employees in the same department
SELECT e1.emp_name AS employee_1,
e2.emp_name AS employee_2,
e1.dept_id
FROM Employee e1
JOIN Employee e2
ON e1.dept_id = e2.dept_id
AND e1.emp_id < e2.emp_id
ORDER BY e1.dept_id, e1.emp_id, e2.emp_id;
+-----------+-----------+---------+
| employee_1 | employee_2 | dept_id |
+-----------+-----------+---------+
Aarav	Priya	1
Aarav	Rahul	1
Aarav	Ananya	1
Aarav	Rohan	1
Aarav	Neha	1
Priya	Rahul	1
Priya	Ananya	1
Priya	Rohan	1
Priya	Neha	1
Rahul	Ananya	1
Rahul	Rohan	1
Rahul	Neha	1
Ananya	Rohan	1
Ananya	Neha	1
Rohan	Neha	1
Aditi	Karan	2
Ready
Light
Feedback
New
History
Save


byteXL
Ctrl + K
PARUL NAMDEV
queries.sql
MySQL
Run
I/O
AI Agent
104 ms
STDIN
Input for the program (Optional)
Output:
-- 4.1 INNER JOIN: employees with their department
SELECT e.emp_id, e.emp_name, e.salary, d.dept_name
FROM Employee e
INNER JOIN Department d ON e.dept_id = d.dept_id;
-- 4.2 LEFT JOIN: all departments, including departments without employees
SELECT d.dept_id, d.dept_name, e.emp_id, e.emp_name
FROM Department d
LEFT JOIN Employee e ON d.dept_id = e.dept_id;
-- 4.3 SELF-JOIN: show employees in the same department
SELECT e1.emp_name AS employee_1,
e2.emp_name AS employee_2,
e1.dept_id
FROM Employee e1
JOIN Employee e2
ON e1.dept_id = e2.dept_id
AND e1.emp_id < e2.emp_id
ORDER BY e1.dept_id, e1.emp_id, e2.emp_id;
-- 4.4 THREE-WAY JOIN: project, employee and department details
SELECT p.project_id, p.project_name, p.budget,
e.emp_name, d.dept_name
FROM Project p
INNER JOIN Employee e ON p.emp_id = e.emp_id
INNER JOIN Department d ON p.dept_id = d.dept_id;
Sana	Naina	5
Sana	Krish	5
Aryan	Naina	5
Aryan	Krish	5
Naina	Krish	5
project_id	project_name	budget	emp_name	dept_name
201	Website Development	100000.00	Aarav	CSE
202	AI Chatbot	150000.00	Aditi	AIML
206	Machine Learning	350000.00	Arjun	AIML
203	Smart Sensor	200000.00	Simran	ECE
204	Cloud Migration	250000.00	Meera	IT
207	Network Security	400000.00	Kabir	IT
205	Data Analytics	300000.00	Diya	DS
208	Big Data System	450000.00	Aryan	DS
Ready
Light


byteXL
Ctrl + K
PARUL NAMDEV
queries.sql
MySQL
Run
I/O
AI Agent
362 ms
STDIN
Input for the program (Optional)
Output:
FROM Project p
INNER JOIN Employee e ON p.emp_id = e.emp_id
INNER JOIN Department d ON p.dept_id = d.dept_id;
-- 4.5 CORRELATED SUBQUERY: employees earning above their department average
SELECT e.emp_id, e.emp_name, e.salary, e.dept_id
FROM Employee e
WHERE e.salary > (
SELECT AVG(e2.salary)
FROM Employee e2
WHERE e2.dept_id = e.dept_id
);
-- 4.6 EXISTS: departments that have at least one employee
SELECT d.dept_id, d.dept_name
FROM Department d
WHERE EXISTS (
SELECT 1
FROM Employee e
WHERE e.dept_id = d.dept_id
);
-- 4.7 NOT EXISTS: departments with no employees
SELECT d.dept_id, d.dept_name
FROM Department d
WHERE NOT EXISTS (
SELECT 1
emp_id	emp_name	salary	dept_id
104	Ananya	36000.00	1
105	Rohan	38000.00	1
106	Neha	40000.00	1
110	Arjun	36000.00	2
111	Isha	38000.00	2
112	Varun	40000.00	2
116	Manav	36000.00	3
117	Pooja	38000.00	3
118	Nikhil	40000.00	3
122	Kabir	36000.00	4
123	Tanya	38000.00	4
124	Yash	40000.00	4
128	Aryan	36000.00	5
129	Naina	38000.00	5
130	Krish	40000.00	5
Ready
Light
Feedback
New
History
Save


byteXL
Ctrl + K
PARUL NAMDEV
queries.sql
MySQL
Run
I/O
AI Agent
362 ms
STDIN
Input for the program (Optional)
Output:
-- 4.6 EXISTS: departments that have at least one employee
SELECT d.dept_id, d.dept_name
FROM Department d
WHERE EXISTS (
SELECT 1
FROM Employee e
WHERE e.dept_id = d.dept_id
);
-- 4.7 NOT EXISTS: departments with no employees
SELECT d.dept_id, d.dept_name
FROM Department d
WHERE NOT EXISTS (
SELECT 1
FROM Employee e
WHERE e.dept_id = d.dept_id
);
-- 4.8 SIMULATED INTERSECT: employees whose department exists in Project
-- MySQL versions before 8.0.31 do not support INTERSECT directly.
SELECT e.emp_id, e.emp_name
FROM Employee e
WHERE EXISTS (
SELECT 1
FROM Project p
WHERE p.emp_id = e.emp_id
);
dept_id	dept_name
1	CSE
2	AIML
3	ECE
4	IT
5	DS
emp_id	emp_name
101	Aarav
107	Aditi
110	Arjun
113	Simran
119	Meera
122	Kabir
125	Diya
Ready
Light
Feedback
New
History
Save


byteXL
Ctrl + K
PARUL NAMDEV
queries.sql
MySQL
Run
I/O
AI Agent
362 ms
STDIN
Input for the program (Optional)
Output:
WHERE NOT EXISTS (
);
-- 4.10 EXPLAIN query plan for INNER JOIN
EXPLAIN
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
INNER JOIN Department d ON e.dept_id = d.dept_id;
-- 4.11 EXPLAIN query plan for correlated subquery
EXPLAIN
SELECT e.emp_id, e.emp_name, e.salary
FROM Employee e
WHERE e.salary > (
SELECT AVG(e2.salary)
FROM Employee e2
WHERE e2.dept_id = e.dept_id
);
-- Optional: create indexes, then compare EXPLAIN output again
CREATE INDEX idx_employee_dept_id ON Employee(dept_id);
CREATE INDEX idx_project_emp_id ON Project(emp_id);
EXPLAIN
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
INNER JOIN Department d ON e.dept_id = d.dept_id;
| id | select_type | table | partitions | type | possible_keys | key
| 1 | SIMPLE | e | NULL | ALL | dept_id | NULL
| 1 | SIMPLE | d | NULL | eq_ref | PRIMARY | PRIMARY
| id | select_type | table | partitions | type | possible_keys | ke
| 1 | PRIMARY | e | NULL | ALL | NULL | NU
| 2 | DEPENDENT SUBQUERY | e2 | NULL | ALL | dept_id | NU
| id | select_type | table | partitions | type | possible_keys | ke
| 1 | SIMPLE | d | NULL | ALL | PRIMARY | NU
| 1 | SIMPLE | e | NULL | ref | idx_employee_dept_id | id
Ready
Light